CHE 565 -- Homework 5

Soki Sem

April 29, 2026

Introduction

The following homework was done with Simulink and the Control Systems Library in Python. The block diagram of the system is shown in Figure 1. The block diagram was created in simulink and then I built the same diagram as a python function using the control library. Simulink was mainly used as a sanity check for the response of the system in the python code.

Note: sum2 in the block diagram is the summing junction that takes the first controller output and subtract the P stream in order to form the inner loop. But, first in order to simulate without the inner loop, so I set cascade=False. This just converts the controller output (Yc1) to the error signal (E2) for the second controller. The disturbance D is added directly to the output of Gp1 (Yp1)

The transport delay transfer function is approximated using a Pade approximation of order 1.Although it should be relatively straightforward to simulate with delay in simulink and in python, I commented through the delay blocks in simulink and in python. The assignment didn't specifiy the delay time. Also, the control library has the delay function, but it only uses the Pade approximation.

Which gives the following transfer function approximation:

$$e^{-s\theta_d} = \frac{-\frac{s\theta_d}{2} + 1}{\frac{s\theta_d}{2} + 1}$$

```python
import control as ct

def build_closed_loop(Kc1, Kc2, tauI1, tauI2, cascade=False):

    s = ct.tf('s')
    t = sp.symbols('t', real=True)
    I1 = 1.0 / tauI1
    I2 = 0
    numD,denD = ct.delay.pade(theta, pade_order)

    Gc1 = Kc1 * (1 + I1 / s)
    Gc2 = Kc2 * (1 + I2 / s)
    Gp1 = Kp1 / (taup1 * s + 1)
    Gp2 = Kp2 / (taup2 * s + 1)
    Gd = ct.tf([1], [1])    # direct disturbance addition
    GD1= ct.tf(numD, denD, name='GD1', inputs='Yp1', outputs='YD')
    GD2= ct.tf(numD, denD, name='GD2', inputs='Yp2', outputs='Y')
    # State Space Representation of the blocks for interconnection
    # Note: the control library's interconnect function works better with state-space models, so we convert the transfer functions to state-space form.
    # xdot = Ax + Bu
    # y = Cx + Du

    Gc1_blk = ct.ss(Gc1, name='Gc1', inputs='E1', outputs='Yc1')
    Gc2_blk = ct.ss(Gc2, name='Gc2', inputs='E2', outputs='Yc2')
    Gp1_blk = ct.ss(Gp1, name='Gp1', inputs='Yc2', outputs='Yp1')
```

```
    Gp2_blk = ct.ss(Gp2, name='Gp2', inputs='P', outputs='Y')
    Gd_blk = ct.ss(Gd, name='Gd', inputs='D', outputs='Yd')      # direct disturbance
    addition
    #GD1_blk = ct.ss(GD1, name='GD1', inputs='Yp1', outputs='YD')
    #GD2_blk = ct.ss(GD2, name='GD2', inputs='Yp2', outputs='Y')
    sum1 = ct.summing_junction(inputs=['Ysp', '-Y'], output='E1', name='Sum1')
    if cascade:
         sum2 = ct.summing_junction(inputs=['Yc1', '-P'], output='E2', name='Sum2')
    else:
        sum2 = ct.summing_junction(inputs=['Yc1'], output='E2', name='Sum2')
    sum3 = ct.summing_junction(inputs=['Yp1', 'Yd'], output='P', name='Sum3')
    control_blks = [Gc1_blk, Gc2_blk]
    plant_blks = [Gp1_blk, Gp2_blk]
    disturbance_blks = [Gd_blk]
    #delay_blks = [GD1_blk, GD2_blk]
    sum_blks = [sum1, sum2, sum3]
    blocks = control_blks + plant_blks + disturbance_blks + sum_blks
    sys = ct.interconnect(
        blocks,
        input=['Ysp', 'D'],
        output=['Y'],
        input_prefix = ["Ysp", "D"],
        output_prefix = ["Y"],
    )
    return sys


def calculate_IAE(t, y, ysp):
    error = ysp - y
    iae = np.trapezoid(np.abs(error), t)
    return iae
def simulate_case(sys, t, ysp_input, d_input):
    U = np.vstack([ysp_input, d_input])
    resp = ct.forced_response(sys, T=t, U=U, squeeze=True,return_states=True)
    return resp.time, resp.outputs, resp.inputs, resp.states
%
```

Problem 1

Closed-loop Response to Step Disturbance

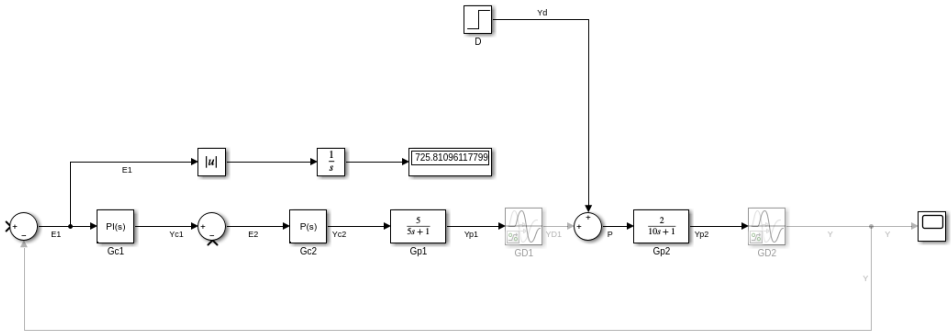
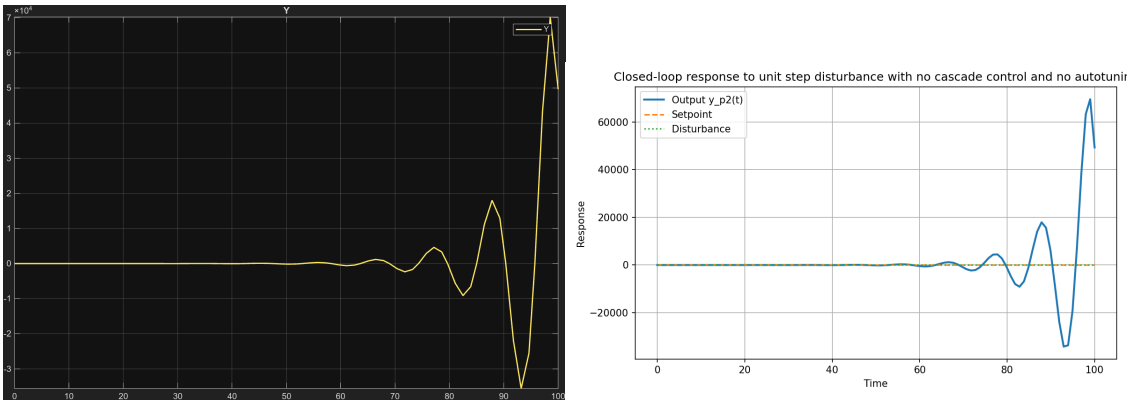


Figure 1: Block diagram of the closed-loop system without cascade control and with a unit step disturbance from Simulink.

The Simulink block diagram is shown in Figure 1.

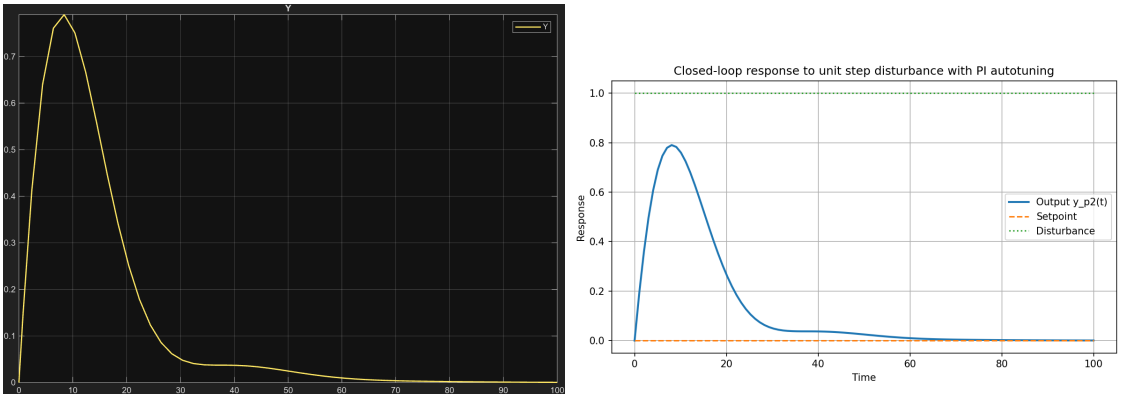


(a) Simulink (b) Python

Figure 2: Closed-loop response to no step change in setpoint and a unit step change in disturbance with no cascade control and no autotuning.

Case 1: Step disturbance with no setpoint change gave and step in disturbance with no setpoint change gave IAE = 449341.5049 in Python and IAE = 453752.63065 in Simulink.The closed-loop disturbance response and no cascade loop is shown in Figure 2.

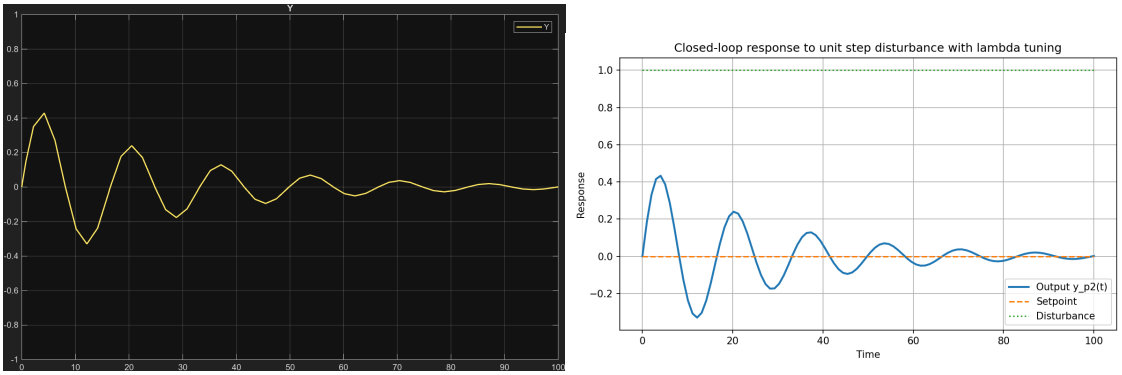
The PI controller was then tuned using the autotuning feature in Simulink,



(a) Simulink (b) Python

Figure 3: Closed-loop response to no step change in setpoint and a unit step change in disturbance with no cascade control and PI autotuning.

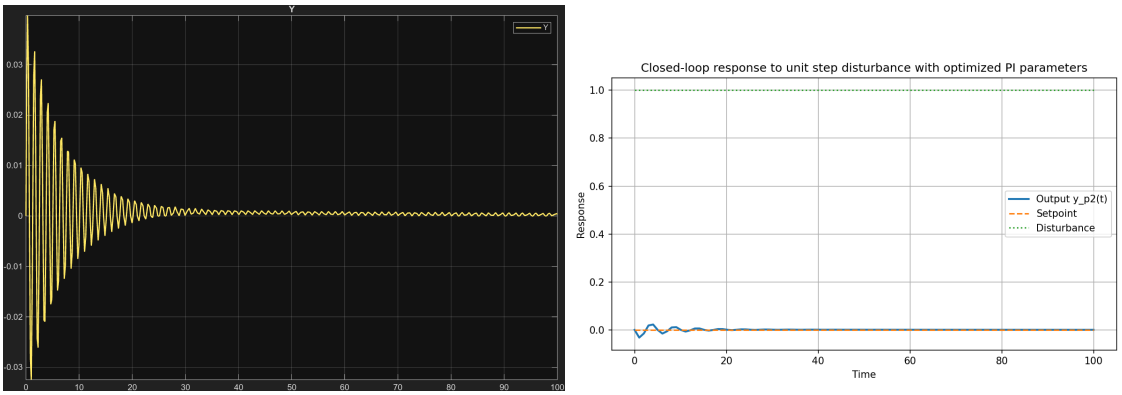
Case 2: Step disturbance with no setpoint change and PI autotuning gave IAE = 13.3102 in Python and IAE = 13.0463 in Simulink. The response is shown in Figure 3. Proportional gain P = 0.1837 and integral gain I = 0.0816.



(a) Simulink (b) Python

Figure 4: Closed-loop response to no step change in setpoint and a unit step change in disturbance with no cascade control and lambda tuning.

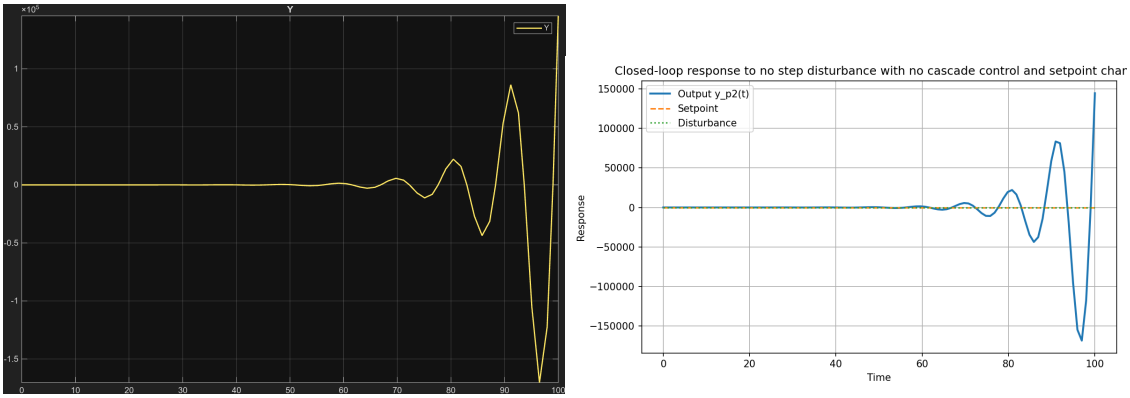
Case 3: No step change in setpoint and step disturbance change with λ tuning gave IAE = 8.5365 in Python and IAE = 8.5578 in Simulink. The disturbance response is shown in Figure 4. Proportional gain P = 0.7031, integral gain I = 0.2308, derivative gain D = 0.0000, and lambda = 1.6667.



(a) Simulink (b) Python

Figure 5: Closed-loop response to unit step disturbance with optimized PI parameters.

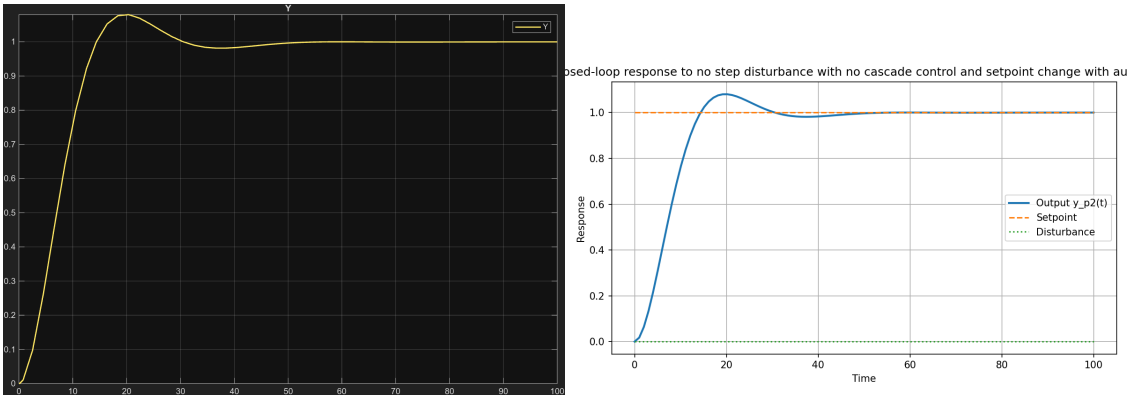
Case 4: Step disturbance with no setpoint change and optimized PI parameters gave IAE = 0.2084 in Python and IAE = 0.2192. The response is shown in Figure 5. Optimized proportional gain P = 123.2445 and integral gain I = 0.0180. Note: The optimized parameters were found by minimizing the IAE to the disturbance



(a) Simulink (b) Python

Figure 6: Closed-loop response to setpoint change with no step disturbance and no cascade control.

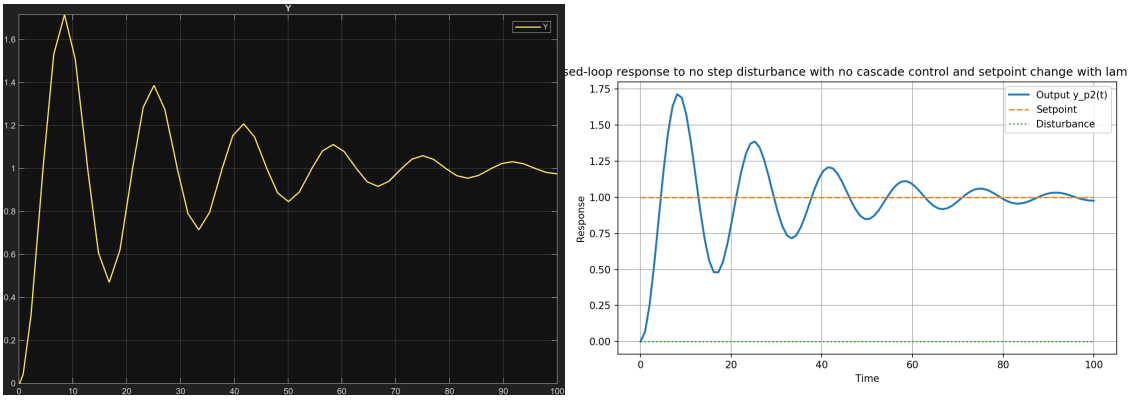
Case 5: No step change in disturbance and step change in setpoint with no cascade control gave IAE = 1230474.8042 in Python and IAE = 1241005.8219 in Simulink. The response is shown in Figure 6.



(a) Simulink (b) Python

Figure 7: Closed-loop response to setpoint change with no step disturbance and no cascade control with PI autotuning.

Case 6: No step change in disturbance and step change in setpoint with no cascade control and PI autotuning gave IAE = 8.2141 in Python and IAE = 8.2136 in Simulink. The response is shown in Figure 7. Proportional gain P = 0.1837 and integral gain I = 0.0816.



(a) Simulink (b) Python

Figure 8: Closed-loop response to setpoint change with no step disturbance and no cascade control with lambda tuning.

Case 7: No step change in disturbance and step change in setpoint with no cascade control and lambda tuning gave IAE = 16.6815 in Python and IAE = 16.6484 in Simulink. The response is shown in Figure 8 . Proportional gain P = 0.7031, integral gain I = 0.2308, derivative gain D = 0.0000, and lambda = 1.6667.